

Java Backend Architecture

Interview Series

Chapter 13.2

Reactive Programming – Project Reactor

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Introduction

Project Reactor is the reactive programming library underlying Spring WebFlux, built on the Reactive Streams specification. The spec defines four interfaces — Publisher, Subscriber, Subscription, and Processor — that together implement a non-blocking, backpressure-aware protocol for async data streams. Reactor provides two primary types: `Mono<T>` for 0-or-1-item streams and `Flux<T>` for 0-to-N streams. These types expose a rich operator vocabulary (`map`, `flatMap`, `filter`, `zip`, `merge`, `concat`, `switchIfEmpty`) that allows complex async pipelines to be expressed declaratively without nested callbacks.

Understanding the cold/hot publisher distinction, scheduler selection (`boundedElastic` for I/O, `parallel` for CPU), and the `subscribeOn/publishOn` threading model is essential for writing correct, performant reactive code. Error handling with `onErrorReturn`, `onErrorResume`, `retry`, and `retryWhen` replaces `try/catch` in the reactive world. Testing reactive pipelines with `StepVerifier` from `reactor-test` gives deterministic, synchronous verification of async sequences. Backpressure handling through `request(n)` and overflow strategies protects downstream consumers from producer overflow.

Key Concepts & Definitions

Publisher<T>	Reactive Streams interface; produces a sequence of items to a Subscriber on subscription.
Subscriber<T>	Reactive Streams interface; receives <code>onNext</code> , <code>onComplete</code> , and <code>onError</code> signals from a Publisher.
Subscription	Reactive Streams interface; represents a link between Publisher and Subscriber. Provides <code>request(n)</code> and <code>cancel()</code> .
Processor<T,R>	Reactive Streams interface that extends both Publisher and Subscriber; acts as a transform stage.
Mono<T>	Reactor type for 0-or-1 asynchronous item. Equivalent to an async Optional or single-value Future.
Flux<T>	Reactor type for 0-to-N asynchronous items with optional completion or error signal.
Cold Publisher	Starts producing items fresh for each subscriber; each subscriber gets the full sequence (e.g., <code>Flux.just()</code>).
Hot Publisher	Shares a single live stream across all subscribers; late subscribers miss earlier items (e.g., <code>Sinks.Many</code>).
flatMap	Operator that maps each element to a Publisher and merges results concurrently (unordered).
switchIfEmpty	Operator that subscribes to an alternate Publisher if the upstream completes empty.
subscribeOn	Sets the Scheduler for the subscription (source execution); affects the upstream chain.
publishOn	Switches the Scheduler for downstream operators; elements emitted on the specified Scheduler.
StepVerifier	<code>reactor-test</code> utility for synchronously testing <code>Mono/Flux</code> sequences step by step.

boundedElastic	Scheduler backed by a bounded, cached thread pool; ideal for blocking I/O calls in reactive pipelines.
retryWhen	Operator for conditional/exponential-backoff retry logic using a companion Flux of retry signals.

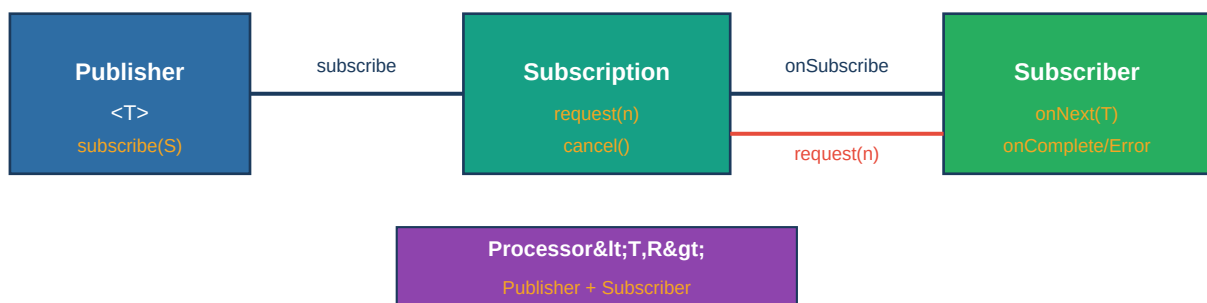
Operator & Strategy Comparison

Operator/Feature	Type	Behavior	Use Case
map	Transform	Sync 1:1 transform	Simple value mapping
flatMap	Transform	Async N:N, merge concurrent	Parallel async calls
concatMap	Transform	Async N:N, preserve order	Sequential async calls
zip	Combine	Pair items from N publishers	Combine parallel results
merge	Combine	Interleave N publishers	Union of streams
concat	Combine	Sequentially chain publishers	Sequential stream union
onErrorReturn	Error	Return fallback value	Simple default on error
onErrorResume	Error	Switch to fallback publisher	Complex recovery logic
retry(n)	Error	Resubscribe up to n times	Transient failure retry
retryWhen	Error	Conditional/backoff retry	Exponential backoff

Architecture Diagrams

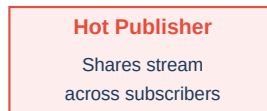
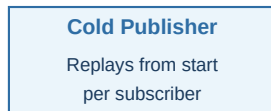
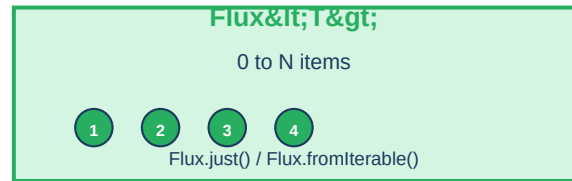
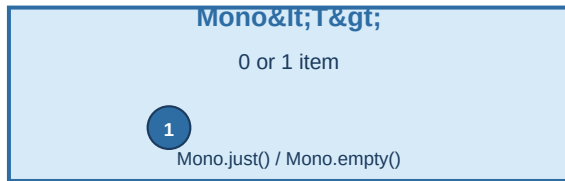
Reactive Streams Specification

Reactive Streams Specification



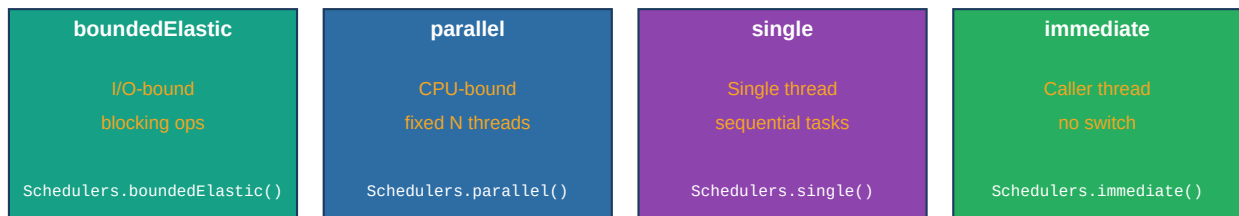
Mono vs Flux — Cold vs Hot Publishers

Mono vs Flux — Project Reactor



Reactor Scheduler Types

Project Reactor — Scheduler Types



subscribeOn() sets source scheduler; publishOn() switches downstream scheduler

Java Code Examples

1. Mono and Flux Basics

```
// Mono — 0 or 1 item Mono<User> userMono = Mono.fromCallable(() ->
userRepo.findById(id)) .subscribeOn(Schedulers.boundedElastic()); // blocking DB call on
boundedElastic // Flux — 0..N items Flux<Product> products =
Flux.fromIterable(productRepo.findAll()) .filter(p -> p.isActive()) .map(p ->
enrichProduct(p)); // Subscribe products.subscribe( p -> log.info("Product: {} ", p.getName()),
err -> log.error("Error", err), () -> log.info("Complete") );
```

2. flatMap vs concatMap

```
Flux<String> userIds = Flux.just("u1", "u2", "u3"); // flatMap: concurrent, unordered
Flux<Order> orders = userIds.flatMap(id -> orderService.getOrdersForUser(id), // each call runs
concurrently 4 // max concurrency = 4 ); // concatMap: sequential, preserves order
Flux<Order> orderedOrders = userIds.concatMap(id -> orderService.getOrdersForUser(id) // waits for each
before subscribing next );
```

3. Error Handling

```
Mono<User> safeUser = userService.findUser(id)
.onErrorReturn(UserNotFoundException.class, User.GUEST) // specific type fallback
.onErrorResume(DatabaseException.class, ex -> // switch publisher on DB error
cacheService.getCachedUser(id)) .timeout(Duration.ofSeconds(3)) .retryWhen(Retry.backoff(3,
Duration.ofMillis(100)) // exp backoff, max 3 retries .filter(ex -> ex instanceof
TransientException));
```

4. subscribeOn vs publishOn

```
Flux.range(1, 10) .subscribeOn(Schedulers.boundedElastic()) // source runs on boundedElastic
.map(i -> heavyTransform(i)) // still on boundedElastic .publishOn(Schedulers.parallel()) //
downstream switches to parallel .map(i -> i * 2) // runs on parallel scheduler
.subscribe(System.out::println); // Rule: subscribeOn affects source & upstream; publishOn
affects everything downstream
```

5. StepVerifier Testing

```
@Test void testUserFlux() { Flux<String> names = userService.getAllUsernames()
.filter(name -> name.startsWith("A")); StepVerifier.create(names) .expectNext("Alice")
.expectNext("Anna") .expectComplete() .verify(Duration.ofSeconds(5)); } @Test void
testMonoError() { Mono<User> failing = userService.findUser(-1L);
StepVerifier.create(failing) .expectError(UserNotFoundException.class) .verify(); }
```

6. Backpressure with Flux Overflow Strategies

```
// Buffer up to 100 items, then error Flux<Event> buffered = eventSource
.onBackpressureBuffer(100, dropped -> log.warn("Dropped: {}", dropped),
BufferOverflowStrategy.DROP_OLDEST); // Drop new items when subscriber is slow Flux<Tick> ticks
= highFrequencySource .onBackpressureDrop(dropped -> metrics.incrementDropped()); // Keep only
latest item when slow Flux<Price> prices = priceStream .onBackpressureLatest(); // Control
demand manually priceStream.subscribe(new BaseSubscriber<Price>() { protected void
hookOnSubscribe(Subscription sub) { request(1); } protected void hookOnNext(Price p) {
process(p); request(1); } // pull 1 at a time });
```

Interview Q&A;

Q1. What is the Reactive Streams specification?

Reactive Streams is a JVM standard (java.util.concurrent.Flow in Java 9+) defining four interfaces: Publisher<T>, Subscriber<T>, Subscription, and Processor<T,R>. It specifies a non-blocking, asynchronous stream protocol with backpressure: the Subscriber controls demand via Subscription.request(n), preventing producers from overwhelming consumers.

Q2. What is the difference between Mono and Flux?

Mono<T> represents 0 or 1 asynchronous item — analogous to an async Optional. Flux<T> represents 0 to N asynchronous items with an optional terminal signal (onComplete or onError). Mono is used for service calls returning a single resource; Flux for queries returning collections or continuous event streams.

Q3. Explain cold vs hot publishers.

A cold publisher starts fresh for each subscriber — each subscriber gets the full sequence independently (like replaying a recording). A cold example is Flux.fromIterable(). A hot publisher emits regardless of subscribers; late subscribers miss earlier items (like a live broadcast). Hot publishers are created via Sinks, ConnectableFlux, or share()/publish().

Q4. What is the difference between flatMap and concatMap?

flatMap subscribes to all inner publishers concurrently and merges results as they arrive — order is not preserved. concatMap subscribes to each inner publisher sequentially, waiting for it to complete before subscribing to the next — order is preserved but slower. Use flatMap for parallel independent calls, concatMap when order matters.

Q5. How does subscribeOn differ from publishOn?

subscribeOn() specifies the Scheduler on which the subscription happens, affecting the source and everything upstream from it. publishOn() inserts a scheduler switch: all operators downstream run on the

specified Scheduler. Only the first `subscribeOn` matters; multiple `publishOn` calls each switch the thread for their downstream portion.

Q6. What Scheduler should you use for blocking I/O in a reactive pipeline?

`Schedulers.boundedElastic()` is designed for blocking I/O. It maintains an elastic but bounded thread pool (default: 10x CPU count with a cap). Use `.subscribeOn(Schedulers.boundedElastic())` to offload blocking JDBC, file I/O, or legacy blocking calls. Never block on `parallel()` or `single()` schedulers as they have very few threads and serve the entire reactive pipeline.

Q7. How do you retry with exponential backoff in Reactor?

Use `.retryWhen(Retry.backoff(maxAttempts, firstBackoff))`. You can filter retryable exceptions with `.filter()`, cap maximum backoff with `.maxBackoff()`, add jitter with `.jitter()`, and handle exhausted retries with `.onRetryExhaustedThrow()`. Example: `Retry.backoff(3, Duration.ofMillis(200)).maxBackoff(Duration.ofSeconds(5)).jitter(0.5)`.

Q8. What is StepVerifier and how do you use it?

`StepVerifier` from `reactor-test` allows synchronous, deterministic testing of `Mono/Flux`. Use `StepVerifier.create(publisher)` to wrap the publisher, then chain `.expectNext(value)`, `.expectNextCount(n)`, `.expectError(ExceptionType.class)`, or `.expectComplete()`, and finally call `.verify()` or `.verify(Duration)` to block until the sequence finishes and assert all expectations.

Q9. Explain the doOnNext and doOnError operators.

`doOnNext(Consumer)` is a side-effect operator that fires a consumer for each emitted item without modifying the stream — useful for logging, metrics, or debugging. `doOnError(Consumer)` fires when the sequence terminates with an error. These do not alter the sequence; they are purely for observability. Equivalent to `peek()` on streams.

Q10. What is the zip operator in Reactor?

`Flux.zip(publisher1, publisher2, ...)` or `Mono.zip(...)` combines items from multiple publishers by pairing the Nth item from each. `zip` waits for all publishers to emit an item before emitting a tuple. It completes when the shortest source completes. Use it to combine parallel independent results, e.g., combining user, orders, and profile fetched concurrently.

Q11. How do you convert a blocking call to a Mono?

Wrap it in `Mono.fromCallable() -> blockingCall().subscribeOn(Schedulers.boundedElastic())`. The `subscribeOn` ensures the blocking lambda runs on the `boundedElastic` pool, not the caller's event-loop thread. Alternatively use `Mono.fromSupplier()` or `Mono.defer() -> Mono.fromCallable(...)` for lazy evaluation.

Q12. What is context propagation in Project Reactor?

The reactive Context is an immutable key-value map attached to each subscription, flowing upstream (opposite to data flow). It replaces `ThreadLocal` for reactive code. You write to it via `contextWrite(ctx -> ctx.put(key, val))` and read via `Mono.deferContextual(ctx -> ...)` or `.transformDeferredContextual()`. Spring Security uses it to propagate `SecurityContext` reactively.

Q13. How does switchIfEmpty work?

`switchIfEmpty(alternativePublisher)` subscribes to the alternative Publisher only if the upstream completes without emitting any items. It is used to provide fallback data sources: e.g., check cache first; if empty, fall back to database. It does not trigger on errors — use `onErrorResume` for error fallback.

Q14. What is the timeout operator in Reactor?

`timeout(Duration)` terminates the sequence with a `TimeoutException` if no item is emitted within the specified duration. You can provide a fallback publisher: `timeout(duration, fallbackPublisher)` to switch to the fallback on timeout instead of erroring. Combined with `retry` or `onErrorResume` it provides resilient, time-bounded reactive calls.

Q15. What is the difference between `merge` and `concat`?

`Flux.merge(p1, p2)` subscribes to all publishers eagerly and interleaves items as they arrive — no order guarantee. `Flux.concat(p1, p2)` subscribes to `p1` first, waits for completion, then subscribes to `p2` — sequential and ordered. `merge` is faster for independent sources; `concat` is needed when order or sequential side-effects matter.

Q16. How do you handle backpressure in a Flux?

Use `onBackpressureBuffer()` to buffer items until the subscriber catches up (with optional size limit and overflow strategy: `DROP_OLDEST`, `DROP_LATEST`, `ERROR`). Use `onBackpressureDrop()` to discard new items when the subscriber is slow. Use `onBackpressureLatest()` to keep only the most recent. For fine control, implement `BaseSubscriber` and call `request(n)` manually.

Q17. What is a `Sinks.Many` and when do you use it?

`Sinks.Many` is the modern (Reactor 3.4+) replacement for `EmitterProcessor` for programmatically emitting items into a Flux from outside the reactive chain (e.g., from a WebSocket handler or event listener). Use `Sinks.many().multicast().onBackpressureBuffer()` for hot multicast, or `replay()` to buffer for late subscribers.

Q18. Explain the reactor-extra library.

`reactor-extra` (`io.projectreactor.addons:reactor-extra`) provides additional operators: `MathFlux` (sum, average, min, max), `TupleUtils` for tuple decomposition, `RetrySpec` extensions, and `Repeat` utilities. It also includes `FlatMapSequential` and `SchedulerAssert` for testing. It is an optional add-on companion to `reactor-core`.

Q19. How does error handling differ between `onErrorReturn` and `onErrorResume`?

`onErrorReturn(fallbackValue)` emits a static fallback value and completes normally when an error occurs — simple and synchronous. `onErrorResume(Function)` switches to a new Publisher (Mono or Flux) when an error occurs — allows async fallback logic (cache lookup, default service call). Both catch errors and allow the stream to complete normally.

Q20. When should you NOT use Project Reactor?

Avoid Reactor for purely CPU-bound work with no I/O — synchronous streams or parallel streams perform better without reactive overhead. Avoid it when the team lacks reactive experience, as debugging reactive stack traces is significantly harder. For simple request/response CRUD with low concurrency, Spring MVC + virtual threads is simpler. Also avoid mixing reactive and blocking code on non-boundedElastic schedulers.